IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 25 August 2025, accepted 15 September 2025,
date of publication 18 September 2025, date of current version 24 September 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3611817

RESEARCH ARTICLE

Investigating the Robustness of Graph Neural Networks to Data Drift: A Case Study on Financial Transaction Data

RENER S. MENEZES, (Graduate Student Member, IEEE), AND RAIMIR H. FILHO, (Member, IEEE)
University of Fortaleza, Fortaleza 60811-905, Brazil

Corresponding author: Rener S. Menezes (rener@edu.unifor.br)

The work of Rener S. Menezes was supported in part by Fitbank Pagamentos Eletronicos SA, for the master's studies.

ABSTRACT The performance of machine learning models in real-world applications is often challenged by data drift, in which the statistical properties of the data evolve over time. This phenomenon is particularly acute in dynamic domains such as financial fraud detection, where adversarial behaviors constantly change. Graph Neural Networks (GNNs), valued for their ability to model complex relational data, are increasingly applied in this domain; however, their inherent robustness to temporal data shifts remains underexplored. This study systematically investigates the impact of natural data drift on the robustness of three foundational GNN architectures: Graph Convolutional Networks (GCNs), Graph Attention Networks (GATs), and Graph Sample and Aggregate (GraphSAGE). Using the large-scale and highly imbalanced IEEE-CIS dataset as a challenging case study, we constructed homogeneous transaction graphs and employed a strict temporal data splitting methodology to simulate a realistic deployment scenario. Model performance was evaluated over 50 sequential, nonoverlapping monitoring windows, using AUC-PR and F2-Score as primary metrics due to their sensitivity in imbalanced contexts. Our findings confirm that all evaluated GNNs suffer from significant performance degradation over time, with GCN and GAT showing pronounced declines. A comparative analysis revealed that GraphSAGE exhibited substantially greater robustness, maintaining more stable and resilient performance. These results highlight that architectural choice is a critical factor for GNNs deployed in dynamic environments and underscore the need to develop adaptive strategies.

INDEX TERMS Data drift, fraud detection, GAT, GCN, graph neural networks, GraphSAGE, IEEE-CIS, imbalanced learning, robustness, temporal analysis.

I. INTRODUCTION

Financial fraud, particularly within the rapidly expanding domain of digital transactions, poses a persistent and increasing threat to global economies, institutions, and individuals alike [1], [2]. Fraudsters continuously devise more sophisticated tactics to circumvent detection systems, leading to substantial annual financial losses and erosion of trust in digital commerce. The sheer volume and velocity of online transactions further compound the detection challenge, necessitating advanced analytical techniques capable of identifying subtle, anomalous patterns indicative of fraudulent behavior.

The associate editor coordinating the review of this manuscript and approving it for publication was Sangsoon Lim.

Two significant hurdles in developing effective fraud detection systems are the inherent **class imbalance** of fraud datasets and the widespread problem of **data drift**. Fraudulent instances are typically rare compared to legitimate transactions, making it difficult for machine learning models to learn discriminative patterns for the minority class without specialized handling [3]. Moreover, the non-stationary nature of financial data means that the statistical properties of data and the characteristics of fraud can change significantly over time, a phenomenon known as data drift [4], [5], [6]. This temporal evolution can render statically trained models obsolete, leading to a decline in predictive performance as new unseen fraud typologies emerge and legitimate user behavior changes [7].

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

Graph Neural Networks (GNNs) have emerged as a powerful paradigm for modeling relational data, making them particularly well suited for financial fraud detection where transactions, users, and associated entities form complex interaction networks [8], [9]. Using graph structures, GNNs can learn rich representations that capture intricate dependencies and neighborhood-based features, potentially uncovering sophisticated fraud schemes that traditional attribute-based models might miss. Architectures such as Graph Convolutional Networks (GCNs) [10], Graph Attention Networks (GATs) [11], and Graph Sample and Aggregate (GraphSAGE) [12] offer distinct mechanisms for aggregating and transforming information from graph-structured data, which is promising in various fraud detection applications [13], [14], [15]. Our study utilizes a homogeneous graph representation where each node corresponds to an individual transaction from the IEEE-CIS dataset, with edges established through multiple relational inference methods to capture diverse inter-transaction dependencies.

Despite advances in GNN-based fraud detection, a critical aspect that requires further investigation is their **robustness to data drift** in real-world, temporally evolving scenarios, especially on large-scale, highly imbalanced datasets, such as the IEEE-CIS benchmark. Although the initial efficacy of these models can be high, their ability to maintain performance as the underlying data distribution changes over time is less understood. This study aims to address this gap by systematically evaluating the impact of natural data drift on the performance of the GCN, GAT, and GraphSAGE models. Our primary objectives are as follows:

1) To quantify the performance degradation of these standard GNN architectures when applied to temporally sequential windows of the IEEE-CIS dataset, using metrics sensitive to imbalanced data, such as the Area Under the Precision-Recall Curve (AUC-PR) and F2-score.
2) To conduct a comparative analysis of the robustness of the GCN, GAT, and GraphSAGE architectures, identifying which models maintained their efficacy better over extended periods of data evolution.

We formulated the following hypotheses to guide our investigation:

- **H1:** Standard GNN models (GCN, GAT, and GraphSAGE) exhibit a significant degradation in fraud detection performance (measured by AUC-PR and F2-score) over time when evaluated on sequential, out-of-distribution data windows of the IEEE-CIS dataset due to natural data drift.
- **H2:** Different GNN architectures (GCN, GAT, and GraphSAGE) demonstrate varying levels of robustness against data drift, resulting in different magnitudes of performance degradation in these temporal windows.

Our experiments involved training the GNN models during the initial period of the IEEE-CIS dataset and then monitoring their performance in subsequent, nonoverlapping temporal windows. The findings of this study provide empirical evidence on the sensitivity of common GNN architectures to data drift in a realistic fraud detection context and offer insights into model selection for dynamic environments.

II. RELATED WORK

The detection of financial fraud has been a significant area of research, evolving from rule-based systems and traditional machine learning (ML) techniques to more sophisticated deep learning (DL) approaches. Although methods such as Logistic Regression, Support Vector Machines, and ensemble models such as Random Forests and Gradient Boosting have been widely applied [16], [17], they often struggle with the highly imbalanced nature of fraud data and the complex and evolving patterns exhibited by fraudsters. More recently, DL models, including autoencoders [18] and Recurrent Neural Networks (RNNs) such as LSTMs [17], [19], have shown promise in the capture of intricate temporal sequences and nonlinearities in transactional data. However, many of these approaches operate primarily on tabular feature representations, potentially overlooking the rich relational information that is inherent in financial networks.

A. GRAPH NEURAL NETWORKS FOR FRAUD DETECTION

Graph Neural Networks (GNNs) have gained substantial attention for their ability to model complex relationships and dependencies within graph-structured data, making them a natural fit for financial fraud detection where entities (e.g., users, accounts, transactions) and their interactions can be represented as graphs [8], [9]. Foundational GNN architectures such as Graph Convolutional Networks (GCNs) [10], Graph Attention Networks (GATs) [11], and GraphSAGE [12] provide powerful mechanisms for learning node representations by aggregating information from their neighborhoods. Several studies have adapted and extended these architectures for fraud detection. For example, CARE-GNN was proposed to improve robustness against camouflaged fraudsters by incorporating label-aware similarity and a reinforcement learning-based neighbor selector [13]. The BRIGHT framework addressed real-time fraud detection challenges by proposing a two-stage directed graph transformation and a Lambda Neural Network (LNN) to manage future information leakage and latency [14]. More recently, FraudGNN-RL combined GNNs with reinforcement learning for adaptive thresholding and feature weighting in response to evolving fraud patterns, achieving high performance on datasets including IEEE-CIS [20]. Although these studies demonstrate the potential of GNNs, many focus on specific types of fraud, utilize private datasets, or propose complex architectures without a systematic comparison of foundational GNNs under natural data drift conditions on publicly available large-scale benchmarks such as IEEE-CIS. Furthermore, beyond local neighborhood aggregation, capturing higher-order structural information is an active area of research, with methods such as Deep Graph Infomax aiming to learn more comprehensive graph representations

by maximizing mutual information between local and global features [31], [32]. Our study complements this body of work by focusing on the fundamental robustness of first-order aggregation models, which serves as an essential baseline for future investigations into more complex architectures.

B. DATA DRIFT AND CONCEPT DRIFT ADAPTATION

Data drift, where the statistical properties of input data change over time, and concept drift, where the relationship between input features and the target variable evolves, are critical challenges in real-world machine learning deployments, especially in adversarial domains such as fraud detection [4], [5], [6], [7]. As noted by Lu et al. [5] and Gama et al. [4], these drifts can severely degrade model performance if not addressed. The literature offers various strategies for drift detection and adaptation, including error rate monitoring, distribution-based methods, and adaptive learning techniques, such as sliding windows, ensemble methods, and model retraining [4], [21]. Sahiner et al. [7] empirically demonstrated the significant impact of data drift on ML models in medical imaging, emphasizing that performance monitoring alone is often insufficient to detect the underlying data changes [22]. Although these studies provide valuable frameworks for understanding and handling data drift, their application and evaluation, specifically within the context of GNN-based financial fraud detection, particularly on large transaction graphs such as those derived from the IEEE-CIS dataset, are less explored. Some GNN-based approaches, such as AT-GCN, which uses LSTMs to adapt GCN parameters over time for Bitcoin transaction graphs [23], and FraudGNN-RL, which employs reinforcement learning for dynamic adaptation [20], [24], incorporate adaptive mechanisms. Similarly, in the domain of spatiotemporal prediction, recent work has focused on designing informative GNNs that distill invariant patterns to improve the accuracy of the prediction under data drift [34]. However, a systematic investigation of the inherent robustness of different standard GNN architectures (GCN, GAT, and GraphSAGE) to natural, unmitigated data drift is required.

C. CLASS IMBALANCE IN GRAPH DATA

The severe class imbalance in fraud data poses a major challenge for GNNs. Standard oversampling techniques like SMOTE (Synthetic Minority Oversampling Technique) are not directly applicable to graph data, as they operate on feature vectors and do not generate relational (edge) information for the new synthetic nodes. This problem has been specifically addressed in the literature. Han et al. [37] proposed Borderline-SMOTE, which focuses on oversampling on minority instances near the class boundary, an idea that can be conceptually extended to graphs. More directly, Zhao et al. [35] introduced **GraphSMOTE**, a seminal framework that generates synthetic nodes in a GNN-based embedding space and trains a dedicated edge generator to predict links for these new nodes, thus creating a balanced and topologically coherent training graph. Other studies have also explored different resampling strategies to handle imbalanced data [36], [38]. A comprehensive review by Mienye and Jere [39] highlights the various algorithms and challenges in this area. Our work acknowledges these advanced methods and implements a pragmatic, heuristic-based graph-aware SMOTE, providing a valuable comparison point for future studies on more complex oversampling techniques in the context of data drift.

D. TEMPORAL VALIDATION AND CHALLENGES IN FRAUD DATASETS

The IEEE-CIS Fraud Detection dataset [25], which is widely used as a benchmark, presents significant challenges, including severe class imbalance, high dimensionality, anonymized features, and inherent non-stationarity [16], [17], [18], [19]. The effective evaluation of such datasets necessitates rigorous temporal validation strategies that mimic real-world deployment scenarios, in which models are trained on past data and predict future potentially drifted data. Solutions from the Kaggle competition for this dataset, such as those of Deotte [26] and CPMP [27], underscore the critical importance of handling temporal aspects, identifying ''client drift'' (changes in the user base) and employing robust time-aware cross-validation schemes. These practical insights highlight that models must generalize not only to unseen transactions but also to evolving underlying data distributions and entity characteristics.

E. POSITIONING THE CURRENT STUDY

Although existing research has advanced GNNs for fraud detection and has addressed data drift separately, there is a gap in understanding the comparative resilience of common GNN architectures when faced with natural data drift in large, imbalanced, and temporally evolving financial transaction datasets. Many studies have focused on proposing novel adaptive mechanisms or achieving high performance on specific (often private or less complex) datasets, without dedicated analysis of how foundational GNNs degrade over time. This study aims to fill this gap by:

1) Empirically evaluating the performance degradation of three widely used GNN architectures (GCN, our GAT implementation, and GraphSAGE) on the IEEE-CIS dataset under conditions of natural data drift using a strict temporal data splitting and monitoring protocol.
2) Conducting a direct comparative analysis of their robustness using metrics (AUC-PR, F2-score) that are critical for real-world imbalanced fraud detection scenarios.
3) Providing insights into the inherent stability of these architectures, which can inform model selection for dynamic fraud detection environments before more complex adaptive mechanisms are considered.

By focusing on the impact of unmitigated drift on standard GNNs, we provide a baseline understanding that is crucial for future research on drift-aware GNNs and adaptive fraud detection systems.

III. METHODOLOGY

This section details the experimental methodology used to investigate the impact of data drift on different GNN architectures for fraud detection using the IEEE-CIS dataset [25]. We describe the dataset, comprehensive data preprocessing pipeline, graph construction process, evaluated GNN models, and the training procedures.

A. DATASET DESCRIPTION

This study used the publicly available IEEE-CIS Fraud Detection dataset from Kaggle [25]. This dataset consists of transactional data, with transactions labeled as fraudulent or legitimate. It is known for its significant class imbalance, high dimensionality, and presence of anonymized features, making it a challenging and realistic benchmark for fraud detection research. The dataset was divided into two main tables, `transaction` and `identity`, which were merged using the `TransactionID` column. The `TransactionDT` feature, which represents a time delta from a reference date, is crucial for temporal analysis and was used to order transactions chronologically for our time-based splitting strategy.

B. DATA PREPROCESSING

A comprehensive preprocessing pipeline was implemented to prepare the IEEE-CIS dataset for GNN modeling, addressing missing values, categorical features, and the need for feature engineering and normalization.

1) MISSING VALUES HANDLING

A multistage approach was adopted to handle missing values across different feature types:

- `V-Features` (Anonymized Numerical): Features with over 90% missing values were removed. Binary indicator flags (e.g., `V_feature_name_missing`) were created for features with more than 5% missingness. Numerical `V-features` with less than 50% missing values were imputed using K-nearest neighbor (KNN) imputation ($k = 5$), while those with 50–90% missingness were imputed using the median. Extreme values (e.g., $> 1 \times 10^{30}$) were flagged and replaced with `NaN` prior to imputation.
- `C-Features` (Numerical Card-related): Missing values were primarily imputed using KNN, falling back to median imputation where KNN was unsuitable. Missing indicator flags are also created.
- `D-Features` (Numerical Debit-related): Similarly to other numerical features, the KNN imputation was used for features with missingness $< 50\%$ and the median imputation for those with higher rates. Missing flags are generated.
- `M-Features` (Match Features): These categorical features had missing values replaced with a distinct 'unknown' category, preserving the information content of missingness.
- `ID-Features`: Missing values were tracked using indicator flags. Categorical `ID-features` were imputed with 'Unknown', and numerical `ID-features` followed the imputation strategy for other numerical variables.

2) CATEGORICAL FEATURE ENCODING

Categorical variables (e.g., `ProductCD`, `card4`, `card6`, `P_emaildomain`, `M1`–`M9`, and `id_12` through `id_38`) had missing values replaced with 'Unknown'. High-cardinality features, such as `card1`, `card2`, and `card3`, were transformed using frequency encoding (e.g., `card1_freq`). `DeviceInfo` was also treated as categorical, with missing values encoded as 'unknown'. Ordinal encoding, which is appropriate for other categorical features, is also applied.

3) FEATURE ENGINEERING AND TRANSFORMATION

Several types of features were engineered to enhance predictive power:

- **Temporal Features:** The `TransactionDT` feature was converted to datetime (reference: 2017-12-01) from which features such as `transaction_hour`, `transaction_dayofweek`, `transaction_month`, `transaction_year`, `transaction_part_of_day`, and `is_weekend` (binary) were derived.
- **Domain-Specific Features:** Features such as `card_velocity`, `card_addr_hash`, `missing_crucial_info` count, and `amt_to_avg_card_ratio` were created.
- **Advanced Transformations:** Log transformations were applied to skewed positive numerals. Interactions between the card/address and the ratio features were also generated.

4) FEATURE SCALING AND NORMALIZATION

To ensure comparable scales and handle outliers, winsorization (1st/99th percentiles) was applied for the outlier treatment. The `RobustScaler` is the primary normalizer for most numerical features. The `StandardScaler` was used during KNN imputation, and the `PowerTransformer` (Yeo-Johnson) was used for skewed distributions.

5) FINAL FEATURE VECTOR CONSTRUCTION

The preprocessing pipeline culminated in 38 features selected for the GNN input, which were guided by statistical analysis and domain knowledge. The set includes the original numerical features (subset of `V` and `C` features, `TransactionAmt`) after imputation/outlier treatment (e.g., `V308`, `C1`), eight missing value indicators (e.g., `D12_missing`), engineered temporal features (e.g., `transaction_hour`), and engineered domain-specific features (e.g., `card_velocity`, `card1_freq`). All 38 features were normalized using `RobustScaler` before the GNN input.

C. GRAPH CONSTRUCTION METHODOLOGY

To leverage GNNs' relational capabilities, the preprocessed tabular data are transformed into a graph in which transactions are nodes. Each node, representing a single transaction from the IEEE-CIS dataset, is homogeneous ("Transaction" type) and is endowed with a 38-dimensional feature vector from preprocessing. Edges representing inter-transaction relationships are bidirectional and established using four complementary methods to capture diverse interaction patterns:

- **Similarity-based Edges:** Transactions linked if the feature vector cosine similarity exceeded 0.8 (max 5 edges/node), capturing substantial shared characteristics. This value was determined through an exploratory analysis of similarity distributions, identifying a point that captured strong feature-based relationships while filtering out weaker, potentially noisy connections.
- **Temporal Proximity Edges:** Transactions within a 2-hour (7200s) window connected, reflecting temporal clustering of fraud (sliding window with 200 transaction limits for efficiency). This duration was chosen as a domain-specific heuristic to model temporally clustered fraudulent activities, a common pattern where fraudsters execute a series of transactions in a short period.
- **Cluster-based Edges:** Transactions clustered via `MiniBatchKMeans` ($k = 3$ from prior analysis, batch size 1024); nodes in the same cluster are interconnected (limit 10k edges/cluster) for structural similarities. This optimal value for k was systematically determined through a preliminary analysis employing the elbow method, validated with the silhouette score and the Calinski-Harabasz index, which consistently suggested three dominant transactional archetypes.
- **Anomaly-based Edges:** Transactions with similar anomalous patterns connected via cosine similarity (>0.75) on up to 5 key anomaly-indicative features (e.g., `V140`, `V261`, identified via "ZScore_max" method), allowing broader matching of unusual patterns (max 3 edges/node). This broader criterion was designed to capture more diverse patterns of unusual behavior that might not be as closely clustered as overtly fraudulent transactions.

Optimizations such as sampling, batch processing, and partial sorting managed the computational demands.

1) GRAPH SCOPE AND STATISTICS

The dataset was partitioned chronologically via `TransactionDT` into four distinct, non-overlapping graph splits (Training, Validation, Test, Monitoring) to prevent data leakage and simulate deployment. Table 1 summarizes the topological characteristics. The statistics for the training graph reflect its state after a graph-aware SMOTE procedure (detailed in "Training Procedure") addresses the class imbalance, altering its original fraud rate from ∼3.33% to ∼13.04%. The validation, test, and monitoring graphs were not subjected to SMOTE. All base graphs (including the original training graph before SMOTE) were single connected components.

TABLE 1. Topological statistics of the generated graphs.

Metric	Training	Validation	Test	Monitoring
Num. Nodes	303,495	58,500	58,500	200,540
Num. Edges	3,175,926	641,288	644,778	1,813,514
Fraud Rate (%)	13.04	3.62	3.70	3.64
Density (10^{-5})	3.45	18.74	18.84	4.51
Avg. Degree	10.46	10.96	11.02	9.04
Approx. Diameter	10	6	6	7
Clustering Coeff.	0.006	0.058	0.057	0.007
Avg. Fraud Node Deg.	19.58	14.44	12.95	10.07
Avg. Non-Fraud Node Deg.	9.10	10.83	10.95	9.00
Fraud Homophily	0.192	0.387	0.408	0.479

D. GRAPH NEURAL NETWORK ARCHITECTURES

Three GNN architectures were evaluated: Graph Convolutional Network (GCN) [10], Graph Attention Network (GAT) [11], and GraphSAGE [12], all implemented using PyTorch [29] and PyTorch Geometric (PyG) [30]. The input of each GNN is a 38-dimensional node feature vector, and the output is a single logit for binary fraud classification. All models consisted of three GNN layers (hidden dimension 128) with ReLU activation and dropout, followed by a final linear layer. The GCN model follows the standard architecture proposed by Kipf and Welling [10]. Its specific architecture is `GCNConv(38,128)` → `ReLU` → `Drp` → `GCNConv(128,128)` → `ReLU` → `Drp` → `GCNConv(128,128)` → `ReLU` → `Drp` → `Lin(128,1)`. The GAT [11] implementation incorporated enhancements for stability and performance, including Layer Normalization (`LayerNorm(affine=True, mode='graph')`) after each `GATConv`, residual connections, a multi-head attention strategy (first two `GATConv` layers 16 heads, concatenated, final `GATConv`: 8 heads, averaged), custom Xavier weight initialization, and edge dropout. Its architecture is as follows: `GAT: GATConv(38,8,h=16)` → `LNorm` → `ReLU` → `Drp` → `GATConv(128,8,h=16)` → `LNorm` → `ReLU` → `Drp` → `GATConv(128,16,h=8)` → `LNorm` → `ReLU` → `Drp` → `Lin(16,1)`. The GraphSAGE model [12] used a `mean` aggregator. Its architecture is `SAGE: SAGEConv(38,128,aggr='mean')` → `ReLU` → `Drp` → `SAGEConv(128,128, aggr='mean')` → `ReLU` → `Drp` → `SAGEConv(128,128,aggr='mean')` → `ReLU` → `Drp` → `Lin(128,1)`.

E. TRAINING PROCEDURE

All GNN models were trained using the Adam optimizer [30], with tuned learning rates and weight decay (GCN/GraphSAGE: $lr = 0.001, wd = 0.0005$; GAT: $lr = 0.0005$). A dropout rate of 0.5 was applied after each GNN layer for GCN/GraphSAGE; GAT utilized internal dropout and edge dropout mechanisms. To address the severe class imbalance, we implemented a two-phase,

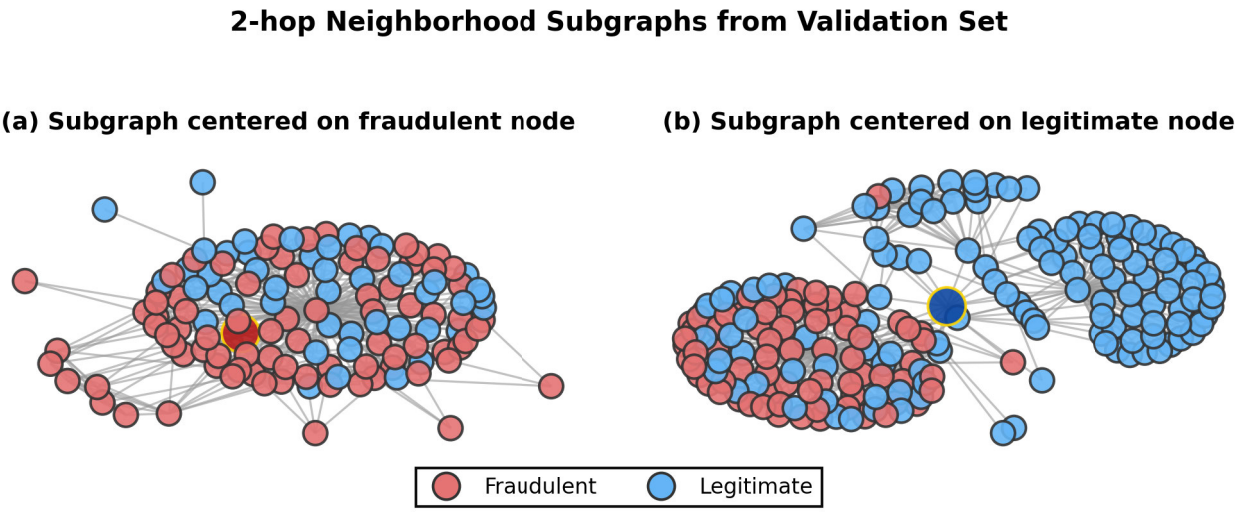


FIGURE 1. Visualization of 2-hop neighborhood subgraphs from the validation set. (a) The subgraph centered on a fraudulent node (red, highlighted) displays clear homophily, forming a dense cluster with other fraudulent nodes. (b) In contrast, the legitimate node's (blue, highlighted) neighborhood is predominantly legitimate. GNNs exploit this structural difference for fraud detection.

graph-aware SMOTE procedure as a preprocessing step on the training graph. Although sophisticated methods such as GraphSMOTE [35] learn a dedicated edge generator, our approach adopts a pragmatic heuristic designed to efficiently preserve the local neighborhood structure of minority instances. Phase 1 consists of applying the standard SMOTE algorithm from the `imbalanced-learn` library to the node feature space, generating synthetic minority (fraud) nodes to achieve a target fraud rate of approximately 15%. Phase 2 integrates these synthetic nodes into the graph topology. For each synthetic node S, edges are strategically created by connecting it to its three most similar original fraudulent nodes and their immediate neighbors. To maintain structural diversity and prevent overfitting to homophilous clusters, connections to two randomly selected non-fraudulent nodes are also added. All new connections are established as bidirectional edges. All GNN models were subsequently trained on the SMOTE-augmented training graph. A weighted binary cross-entropy loss (`BCEWithLogitsLoss`) was employed, with the positive class weight (`pos_weight`) automatically calculated based on the inverse class frequency of augmented training data. The models were trained using a full-graph batch processing approach (the entire graph per split was processed in a single pass per epoch). Training was conducted for up to 100 epochs with an early stopping mechanism: GCN and GraphSAGE monitored the AUC-PR in the validation set (patience 30), and GAT monitored the F2-score in validation (patience 20). All the experiments were conducted on an NVIDIA Tesla T4 GPU with a fixed random seed (42) for reproducibility.

F. EVALUATION STRATEGY FOR DATA DRIFT ANALYSIS

1) TEMPORAL DATA SPLITTING

A strict temporal splitting strategy partitions the IEEE-CIS dataset, ensuring distinct, chronologically ordered training, validation, testing, and monitoring data to prevent leakage and simulate production deployment. The splits were defined by the normalized time of `TransactionDT` as follows: training ($[-0.8664, -0.0822)$), validation ($[-0.0822, 0.1017)$), test ($[0.1017, 0.3003)$), and monitoring ($[0.3003, 1.0126]$).

2) MONITORING PROTOCOL

After the training and initial test set evaluation, the model performance was monitored over 50 sequential, non-overlapping windows (3000 transactions each, ∼1 day's data) from the monitoring graph. This allowed for the observation of performance trends as the models encountered progressively newer data, simulating natural data drift.

3) EVALUATION METRICS

Given a severe class imbalance, the metrics are sensitive to the minority (fraud) class: The Area Under the Precision-Recall Curve (AUC-PR); F2-Score (with $\beta = 2$, emphasizing recall: $F_2 = (1 + \beta^2)\frac{\text{precision} \times \text{recall}}{(\beta^2 \times \text{precision}) + \text{recall}}$); and Recall (Sensitivity). The Area Under the ROC Curve (AUC-ROC) is also reported for completeness. The AUC-PR and F2-score were primarily used to assess performance degradation due to data drift.

IV. RESULTS

This section presents the empirical results of the study. We first report the fraud detection performance of the GCN, GAT, and GraphSAGE models on a held-out test set. Subsequently, we analyzed their performance trends over 50 sequential monitoring windows, each representing approximately one day of transaction data, to evaluate their robustness against natural data drifts.

A. INITIAL MODEL PERFORMANCE

Before assessing the impact of data drift, the GCN, GAT, and GraphSAGE models were trained and evaluated in temporally

TABLE 2. Initial performance of GNN models on the test set.

Model	Recall	F2-Score	AUC-PR	AUC-ROC
GCN	0.9321	0.8007	0.8253	0.9877
GAT	0.8794	0.7025	0.6956	0.9704
GraphSAGE	0.8637	0.8485	0.9099	0.9952

distinct training, validation, and test sets, as described in Section III. The performance of the test set, which represents the data immediately after the validation period, established the baseline efficacy of each model. Table 2 summarizes the key performance metrics of each architecture.

All three models achieved strong initial performance, with GraphSAGE demonstrating particularly high AUC-PR and F2-Score, followed by the GCN. The GAT model also satisfied predefined performance targets. These results indicate that, prior to significant data drift, GNN architectures are capable of effectively detecting fraudulent transactions in the IEEE-CIS dataset.

B. PERFORMANCE UNDER DATA DRIFT

To investigate the impact of natural data drift (H1) and the comparative robustness of GNN architectures (H2), the trained models were evaluated on 50 sequential, non-overlapping monitoring windows. Each window, containing 3000 transactions, represents approximately one day of transaction data from the monitoring period. The trends for the F2-Score and AUC-PR are presented in Fig. 2 and Fig. 3, respectively, to facilitate a direct comparison of the models' robustness.

A visual inspection of these trends reveals that all models experience a decrease in performance over time, especially in the F2-Score and AUC-PR, which are critical for an imbalanced fraud detection. Although generally high, the AUC-ROC metric exhibits lower sensitivity to performance degradation in the minority fraud class. To quantify degradation, Table 3 compares the F2-Score and AUC-PR for Window 1 (representing the first day of monitoring) with Window 50 (representing performance after approximately 50 days of sustained exposure to data drift).

The results in Table 3 clearly quantify the performance decay. The GCN model experienced a 28.27% drop in the F2-Score and a substantial 40.05% drop in AUC-PR in the 50th monitoring window (approximately 50 days). The GAT model showed a 24.85% decrease in F2-Score and a 37.25% decrease in AUC-PR. In contrast, GraphSAGE exhibited significantly more resilience, with its F2-Score decreasing by only 8.88% and its AUC-PR decreasing by 9.32% over the same period. This supports H1, indicating that GNN models are susceptible to performance degradation due to data drift.

C. COMPARATIVE ROBUSTNESS OF GNN ARCHITECTURES

The trend plots (Fig. 2 and 3) and degradation summary (Table 3) allow a direct comparison of the robustness of the GNN architectures, addressing Hypothesis H2. GraphSAGE consistently maintained higher F2-Score and AUC-PR values throughout the 50 daily monitoring windows than GCN and GAT. Its degradation was markedly less severe and its performance was less volatile. For example, GraphSAGE's AUC-PR remained above 0.88, even in Window 50 (Day 50), whereas that of GCN dropped to 0.56, and that of GAT dropped to 0.51. Despite its architectural enhancements aimed at stability, The GAT model displayed considerable fluctuations and a significant overall decline, particularly in the AUC-PR. Although its F2-score degradation was slightly lower than that of GCN by Window 50, its AUC-PR showed a large drop and its trend line indicated greater instability throughout the monitoring period. GCN demonstrated a more gradual but substantial degradation, particularly in AUC-PR. Its F2-score also showed a consistent downward trend. These findings suggest that under the conditions of natural data drift present in the IEEE-CIS dataset over a monitored period of approximately 50 days, GraphSAGE exhibits superior robustness compared to GCN and the GAT architecture. Differences in their aggregation mechanisms and how they process neighborhood information are likely to contribute to the varying levels of resilience.

V. DISCUSSION

The empirical results presented in Section IV provide strong support for our initial hypotheses and offer valuable insights into the behavior of different GNN architectures under natural data drift conditions in the context of financial fraud detection on the IEEE-CIS dataset.

A. INTERACTION BETWEEN CLASS IMBALANCE, OVERSAMPLING, AND DATA DRIFT

A crucial aspect of this study lies at the intersection of its two primary challenges: class imbalance and data drift. While techniques like SMOTE are essential for enabling a GNN to learn from the minority class, their role in a non-stationary environment deserves critical consideration. This study focuses on the impact of natural data drift on GNN robustness. Techniques such as SMOTE and GraphSMOTE primarily address class imbalance under (quasi-)stationary assumptions [35] and do not, by themselves, mitigate drift. Drift mitigation typically requires explicit detection and adaptation mechanisms such as windowing, forgetting, or dedicated drift detectors [4], [5]. Moreover, re-using old minority examples through oversampling may even hinder adaptation under drift by reinforcing outdated fraud patterns [40], [41]. This tension highlights the complexity of building resilient fraud detection systems.

B. CONFIRMATION OF HYPOTHESES AND IMPACT OF DATA DRIFT

Our findings unequivocally confirm H1: GNN models exhibit significant performance degradation when trained on an initial static dataset and subsequently exposed to temporally evolving data. As shown in Fig. 2 and 3 and quantified in Table 3, all three evaluated architectures—GCN, GAT, and GraphSAGE—experienced a decline in their F2-Score and AUC-PR over 50 daily monitoring windows. This

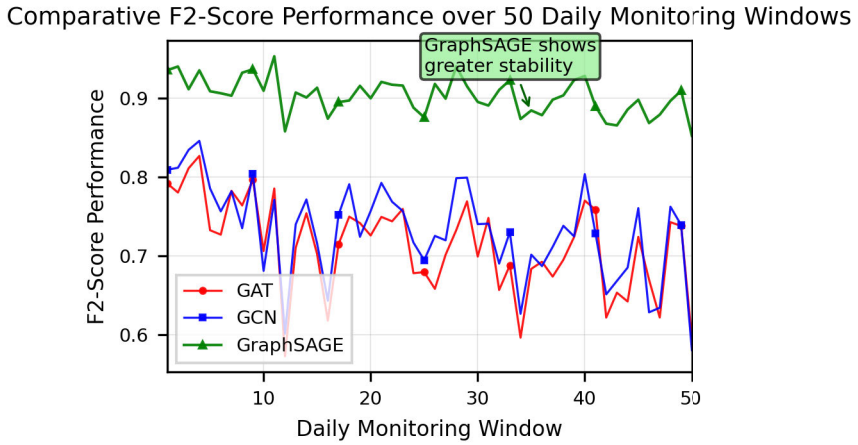


FIGURE 2. Comparative F2-Score performance over 50 daily monitoring windows. GraphSAGE demonstrates significantly greater stability compared to the pronounced degradation of GCN and GAT.

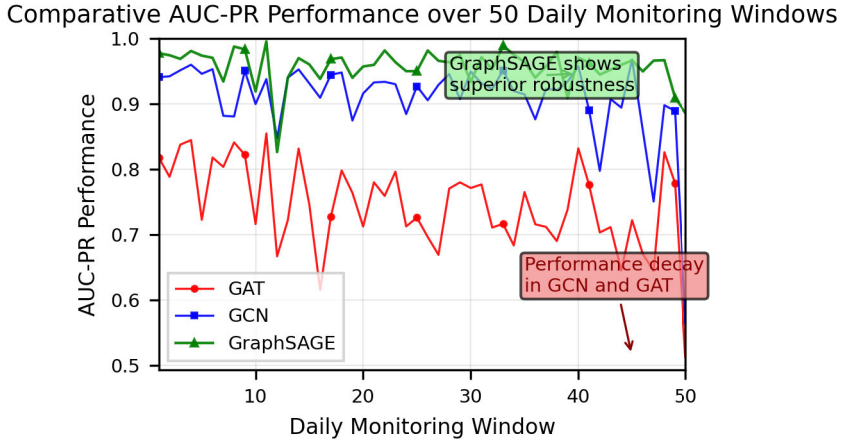


FIGURE 3. Comparative AUC-PR performance over 50 daily monitoring windows. The trend clearly shows the superior robustness of GraphSAGE, while GCN and GAT exhibit substantial performance decay.

TABLE 3. Performance degradation over 50 monitoring windows (approximately 50 days of transactions).

Model	F2-Score			AUC-PR		
	Window 1 (D1)	Window 50 (D50)	% Change	Window 1(D1)	Window 50 (D50)	% Change
GCN	0.8089	0.5802	−28.27%	0.9408	0.5640	−40.05%
GAT	0.7915	0.5948	−24.85%	0.8180	0.5133	−37.25%
GraphSAGE	0.9351	0.8521	−8.88%	0.9775	0.8864	−9.32%

degradation was particularly pronounced for GCN and GAT, with AUC-PR decreasing by 40.05% and 37.25%, respectively. This underscores the critical challenge posed by data drift in real-world fraud detection systems, where models trained on historical data may become less effective as fraud patterns and legitimate transaction behaviors evolve. The relative stability of the AUC-ROC metric across all models, despite significant drops in the AUC-PR and F2-Score, further highlights the importance of selecting evaluation metrics that are sensitive to performance in the minority class in highly imbalanced scenarios, such as fraud detection. H2, which posits that different GNN architectures would exhibit varying levels of robustness against data drift, is strongly supported by our results. GraphSAGE demonstrated markedly superior resilience, with significantly smaller decreases in the F2-Score (−8.88%) and AUC-PR (−9.32%) compared to GCN and GAT. This finding is crucial because it suggests that the architectural choices of GNNs can inherently influence their stability in dynamic environments.

C. ARCHITECTURAL DIFFERENCES AND ROBUSTNESS TO DRIFT

The observed differences in robustness can be attributed to the distinct neighborhood aggregation and information processing mechanisms of each GNN architecture:

GraphSAGE: The superior stability of GraphSAGE, particularly when using a `mean` aggregator as in our experiments, can be linked to its inductive learning capability and its approach to sampling and aggregating features from a fixed-size set of neighbors. By learning aggregator functions that generalize to unseen nodes (or, in our case, evolving node features and local graph structures over time), GraphSAGE may be less susceptible to overfitting to the specific structural patterns of the initial training graph. The `mean` aggregator, by averaging neighbor features, might provide a more smoothed and stable representation that is less perturbed by minor changes in individual neighboring transactions, thus offering better generalization to drifted data.

GCN: GCNs aggregate information from all direct neighbors normalized by the node degree. Although effective in capturing homophilous relationships, this global aggregation across the entire neighborhood may make GCNs more sensitive to broader shifts in the local graph topology or the feature distributions of larger groups of neighbors as data drifts. The substantial degradation observed, particularly in the AUC-PR, suggests that learned GCN filters may not generalize to the evolving characteristics of fraudulent and legitimate transactions over time.

GAT: Despite incorporating enhancements such as LayerNorm and residual connections for training stability, GAT implementation exhibited considerable performance volatility and significant degradation. The attention mechanism in GAT, which assigns different weights to different neighbors, is powerful in capturing complex relationships. However, this flexibility might render it more sensitive to subtle changes in feature distributions or relational patterns caused by data drift. If the characteristics that the attention mechanism learns to prioritize in the training data change significantly in later monitoring windows, the model's ability to identify relevant neighbors and make accurate predictions can be severely hampered, leading to observed instability. The higher number of parameters and complexity often associated with attention mechanisms could also contribute to a greater risk of overfitting the initial data distribution, making generalization to drifted data more challenging.

D. PRACTICAL IMPLICATIONS FOR DEPLOYING GNNS IN FRAUD DETECTION

The findings of this study have several practical implications for the deployment and maintenance of GNN-based fraud detection systems:

1) **Model Selection is Critical:** The choice of GNN architecture is not merely a matter of initial predictive power but also of long-term robustness. For dynamic environments prone to data drift, architectures such as GraphSAGE, which demonstrated greater stability in our study, may be preferred over more complex or globally dependent models such as GAT or GCN, even if the initial performance differences are marginal.
2) **Continuous Monitoring is Essential:** The significant performance degradation observed highlights the necessity of continuous monitoring of model performance in production using appropriate metrics (AUC-PR and F2-Score). Relying solely on AUC-ROC can be misleading.
3) **Proactive Drift Management Needed:** Static models are unlikely to maintain optimal performance. Strategies for detecting data drift (in features or graph structures) and adapting models (e.g. periodic retraining, online learning, or more sophisticated drift mitigation techniques) are crucial for long-term efficacy.
4) **Beware of Over-Optimization on Static Data:** Achieving state-of-the-art results on a static test set does not guarantee sustained performance in a live evolving environment. The robustness to temporal changes is a key consideration in model development and evaluation.

E. LIMITATIONS OF THE STUDY

Although this study provides valuable insights, certain limitations should be acknowledged:

1) **Homogeneous Graph Representation:** Our analysis was conducted on homogeneous graphs, where all nodes represent transactions. Exploring heterogeneous graphs that explicitly model different entities (e.g., cardholders, merchants, and devices) and their diverse relationships could provide richer representations and potentially different robustness characteristics.
2) **Nature of Drift in IEEE-CIS:** The observed data drift is specific to the IEEE-CIS dataset. Although this represents real-world transactional data, the types and rates of drift may vary across different financial systems and fraud domains.
3) **Specific GNN Implementations:** We evaluated specific implementations and hyperparameter settings for GCN, GAT, and GraphSAGE. Although efforts have been made to optimize these, other configurations or variants may yield different robustness profiles.
4) **Passive Drift Observation:** This study focused on observing and quantifying the impact of unmitigated data drift. We did not implement or evaluate the active drift detection or adaptation mechanisms, which is an important next step.
5) **Feature Set:** This study relied on a 38-feature set derived from the original dataset. The specific characteristics of this feature set and its evolution over time influence the observed drift and the model responses.
6) **Graph Oversampling Heuristic:** While our heuristic-based graph-aware SMOTE proved effective for this study, we acknowledge that a direct performance benchmark against state-of-the-art

methods with learned edge predictors, such as GraphSMOTE, was not conducted. Such a comparison remains a valuable direction for future work to quantify the trade-offs between heuristic and learned oversampling strategies in the context of data drift.

F. FUTURE WORK AND IMPROVEMENTS

Building on the findings of this study, several avenues for future work are apparent:

1) **Heterogeneous Graph Models:** Extending this analysis to heterogeneous GNNs could provide a more nuanced understanding of fraud by explicitly modeling different entity types and their interactions, potentially improving both performance and robustness.
2) **Drift Mitigation Strategies:** Investigating and implementing active data drift detection and mitigation techniques specifically for GNN-based fraud detection is a critical next step. This could include adaptive learning methods, periodic retraining strategies informed by drift detectors, or the application of Reinforcement Learning to dynamically adjust the model parameters or decision thresholds in response to evolving data.
3) **Interpretability of Drift Impact:** Developing methods to interpret *why* and *how* data drift affects specific GNN architectures could lead to more targeted robustness enhancements. Understanding the features or graph structures most affected by drift is highly beneficial.
4) **Broader Architectural Comparison:** Expanding the comparative analysis to include other GNN architectures, such as Graph Transformers or temporal GNNs designed explicitly for dynamic graphs, would provide a more comprehensive view of the landscape.
5) **Real-world Deployment and Feedback Loops:** Validating these findings in a live production environment using real-time feedback loops is the ultimate test for model robustness and adaptability.
6) **Sensitivity Analysis of Graph Hyperparameters:** Furthermore, a systematic sensitivity analysis of the graph construction hyperparameters could provide deeper insights into their impact on model performance and robustness, complementing the rationale-based approach taken in this study.

VI. CONCLUSION

This paper presents a comprehensive empirical study on the impact of natural data drift on the performance of three prominent Graph Neural Network architectures—GCN, GAT, and GraphSAGE—for financial fraud detection on the challenging IEEE-CIS dataset. By employing a rigorous temporal splitting methodology and monitoring performance over 50 daily transaction windows, our study provides critical insights into the robustness of these models in dynamic environments. Our primary contribution is the empirical validation that standard GNN models, despite their strong initial performance (e.g., GraphSAGE achieving an initial AUC-PR of 0.9099 and an F2-Score of 0.8485), are susceptible to significant performance degradation when faced with evolving data distributions. This was evidenced by substantial decreases in the F2-Score and AUC-PR for GCN (-28.27% and -40.05%, respectively) and GAT (-24.85% and -37.25%, respectively) by the 50th monitoring day. These findings confirm our first hypothesis (H1) and highlight the perils of deploying static GNN models without continuous monitoring and adaptation strategies in real-world fraud detection systems. Furthermore, this research substantiates our second hypothesis (H2) by demonstrating that the choice of the GNN architecture significantly influences its resilience to data drift. GraphSAGE consistently outperformed GCN and GAT in terms of stability, exhibiting the least performance degradation (F2-Score drop of 8.88%, AUC-PR drop of 9.32%), and maintained higher overall scores on metrics crucial for imbalanced classification. This suggests that GraphSAGE's inductive learning capabilities and its neighborhood sampling and aggregation mechanisms may offer inherent advantages in adapting to or mitigating the effects of evolving data patterns. By contrast, GAT, even with enhancements, showed considerable volatility, indicating that its attention mechanism may be more sensitive to shifts in feature importance or relational structures over time. The practical implications of these findings are significant for practitioners deploying GNNs in fraud detection. This study underscores the necessity of prioritizing architectural robustness alongside initial predictive accuracy, implementing continuous performance monitoring with appropriate metrics (AUC-PR and F2-Score), and planning proactive drift management. Future research will focus on extending this work in several promising directions. First, exploring the use of heterogeneous graph models to incorporate richer entities and relational information can enhance both the detection accuracy and robustness. Second, it is crucial to investigate active drift mitigation techniques, such as the integration of reinforcement learning agents to dynamically adapt model parameters or decision thresholds. Third, developing methods to interpret the impact of data drift on GNNs is crucial to building more resilient and reliable systems. Finally, a broader comparison including other advanced GNN architectures, such as Graph Transformers and models specifically designed for temporal graphs, will further enrich our understanding of GNN behavior in dynamic fraud landscapes.

ACKNOWLEDGMENT

The authors acknowledge the University of Fortaleza for fostering a supportive academic environment that was conducive to this study.

REFERENCES

[1] Federal Trade Commission, Bureau of Consumer Protection. (Mar. 2024). *New FTC Data Show a Big Jump in Reported Losses to Fraud to $12.5 Billion in 2024*. [Online]. Available: https://www.ftc.gov/news-events/news/press-releases/2025/03/new-ftc-data-show-big-jump-reported-losses-fraud-125-billion-2024

[2] A. Reurink, "Financial fraud: A literature review," *J. Econ. Surv.*, vol. 32, no. 5, pp. 1292–1325, Dec. 2018.

[3] Y. Chen, C. Zhao, Y. Xu, C. Nie, and Y. Zhang, "Year-over-year developments in financial fraud detection via deep learning: A systematic literature review," 2025, *arXiv:2502.00201*.
[4] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surveys*, vol. 46, no. 4, pp. 1–37, Apr. 2014.
[5] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.
[6] R. Rastogi. (May 2024). *Model Drift: The Ultimate Guide to Maintaining Machine Learning Model Performance*. [Online]. Available: https://buzzclan.com/data-engineering/what-is-model-drift
[7] B. Sahiner, W. Chen, R. K. Samala, and N. Petrick, "Data drift in medical machine learning: Implications and potential remedies," *Brit. J. Radiol.*, vol. 96, no. 1150, Oct. 2023, Art. no. 20220878.
[8] D. Cheng, Y. Zou, S. Xiang, and C. Jiang, "Graph neural networks for financial fraud detection: A review," *Frontiers Comput. Sci.*, vol. 19, no. 9, Sep. 2025, Art. no. 199609.
[9] E. Dai, T. Zhao, H. Zhu, J. Xu, Z. Guo, H. Liu, J. Tang, and S. Wang, "A comprehensive survey on trustworthy graph neural networks: Privacy, robustness, fairness, and explainability," *Mach. Intell. Res.*, vol. 21, no. 6, pp. 1011–1061, Dec. 2024.
[10] T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.
[11] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lió, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.
[12] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 1024–1034.
[13] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2020, pp. 315–324.
[14] M. Lu, Z. Han, S. X. Rao, Z. Zhang, Y. Zhao, Y. Shan, R. Raghunathan, C. Zhang, and J. Jiang, "BRIGHT—Graph neural networks in real-time fraud detection," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2022, pp. 3342–3351.
[15] J. Lin, X. Guo, Y. Zhu, S. Mitchell, E. Altman, and J. Shun, "FraudGT: A simple, effective, and efficient graph transformer for financial fraud detection," in *Proc. 5th ACM Int. Conf. AI Finance*, Nov. 2024, pp. 292–300.
[16] W. Deng, Z. Huang, J. Zhang, and J. Xu, "A data mining based system for transaction fraud detection," in *Proc. IEEE Int. Conf. Consum. Electron. Comput. Eng. (ICCECE)*, Jan. 2021, pp. 542–545.
[17] O. I. Obaid and A. Y. Al-Sultan, "Leveraging LSTM and attention for high-accuracy credit card fraud detection," *Fusion, Pract. Appl.*, vol. 17, no. 1, pp. 209–220, 2025.
[18] S. Jiang, R. Dong, J. Wang, and M. Xia, "Credit card fraud detection based on unsupervised attentional anomaly detection network," *Systems*, vol. 11, no. 6, p. 305, Jun. 2023.
[19] I. A. Umaru, A. A. Aliyu, M. Ibrahim, S. Abdulkadir, M. A. Ahmed, M. A. Abubakar, A. A. Ahmed, and S. A. Tanko, "An enhanced hybrid model combining LSTM, ResNet, and an attention mechanism for credit card fraud detection," *FUDMA J. Sci.*, vol. 9, no. 2, pp. 42–48, Feb. 2025.
[20] Y. Cui, X. Han, J. Chen, X. Zhang, J. Yang, and X. Zhang, "FraudGNN-RL: A graph neural network with reinforcement learning for adaptive financial fraud detection," *IEEE Open J. Comput. Soc.*, vol. 6, pp. 426–437, 2025.
[21] R. Müller, M. Abdelaal, and D. Stjelja, "Open-source drift detection tools in action: Insights from two use cases," in *Proc. Int. Conf. Big Data Anal. Knowl. Discovery*, 2024, pp. 346–352.
[22] A. Kore, E. Abbasi Bavil, V. Subasri, M. Abdalla, B. Fine, E. Dolatabadi, and M. Abdalla, "Empirical data drift detection experiments on real-world medical imaging data," *Nature Commun.*, vol. 15, no. 1, p. 1887, Feb. 2024.
[23] W. Gu, M. Sun, B. Liu, K. Xu, and M. Sui, "Adaptive spatio-temporal aggregation for temporal dynamic graph-based fraud risk detection," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Feb. 2021, pp. 1094–1103.
[24] Y. Dong, J. Yao, J. Wang, Y. Liang, S. Liao, and M. Xiao, "Dynamic fraud detection: Integrating reinforcement learning into graph neural networks," in *Proc. 6th Int. Conf. Data-driven Optim. Complex Syst. (DOCS)*, Aug. 2024, pp. 818–823.
[25] IEEE Computational Intelligence Society and Vesta Corporation. (2019). *IEEE-CIS Fraud Detection*. [Online]. Available: https://www.kaggle.com/c/ieee-fraud-detection
[26] C. Deotte. (Oct. 2019). *1st Place Solution—Part 1*. [Online]. Available: https://www.kaggle.com/competitions/ieee-fraud-detection/discussion/111284
[27] CPMP. (Oct. 2019). *2nd Solution, CPMP View*. [Online]. Available: https://www.kaggle.com/competitions/ieee-fraud-detection/discussion/111502
[28] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 8026–8037.
[29] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch geometric," in *Proc. ICLR Workshop*, 2019.
[30] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, 2014.
[31] P. Veličković, W. Fedus, W. L. Hamilton, P. Lió, Y. Bengio, and R. D. Hjelm, "Deep graph infomax," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.
[32] Z. Wang, C. Pan, Y. Wang, and Z. Wang, "Incorporating higher-order structural information for graph clustering," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 10, pp. 4975–4987, Oct. 2022.
[33] F. Yang, F. Zhong, X. Wei, and G. Zeng, "A fine-grained defect prediction method based on drift-immune graph neural networks," *Comput., Mater. Continua*, vol. 82, no. 2, pp. 3563–3590, 2025.
[34] J. Zheng, D. Saxena, J. Cao, H. Yang, and P. Ruan, "Inductive spatial temporal prediction under data drift with informative graph neural network," in *Proc. Int. Conf. Database Syst. Adv. Appl.*, 2024, pp. 169–185.
[35] T. Zhao, X. Zhang, and S. Wang, "GraphSMOTE: Imbalanced node classification on graphs with graph neural networks," in *Proc. 14th ACM Int. Conf. Web Search Data Mining*, Mar. 2021, pp. 833–841.
[36] G. E. A. P. A. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *ACM SIGKDD Explor. Newslett.*, vol. 6, no. 1, pp. 20–29, Jun. 2004.
[37] H. Han, W. Wang, and B. Mao, "Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning," in *Proc. Int. Conf. Intell. Comput.*, 2005, pp. 878–887.
[38] G. Husain, D. Nasef, R. Jose, J. Mayer, M. Bekbolatova, T. Devine, and M. Toma, "SMOTE vs. SMOTEENN: A study on the performance of resampling algorithms for addressing class imbalance in regression models," *Algorithms*, vol. 18, no. 1, p. 37, Jan. 2025.
[39] I. D. Mienye and N. Jere, "Deep learning for credit card fraud detection: A review of algorithms, challenges, and solutions," *IEEE Access*, vol. 12, pp. 96893–96910, 2024, doi: 10.1109/ACCESS.2024.3426955.
[40] E. Spyromitros-Xioufis, M. Spiliopoulou, G. Tsoumakas, and I. Vlahavas, "Dealing with concept drift and class imbalance in multi-label stream classification," in *Proc. 22nd Int. Joint Conf. Artif. Intell. (IJCAI)*, 2011, pp. 1583–1588.
[41] E. Aguiar, P. M. T. F. Ferreira, and J. Gama, "A survey on imbalanced data stream classification," *Mach. Learn.*, vol. 113, no. 5, pp. 2223–2281, 2024.



RENER S. MENEZES (Graduate Student Member, IEEE) received the technical degree in information systems from the Faculdade Integrada do Ceará. He is currently pursuing the master's degree with the University of Fortaleza, Fortaleza, Brazil. His current research interests include graph neural networks, data drift, and financial fraud detection.



RAIMIR H. FILHO (Member, IEEE) received the Ph.D. degree in computer science from the Universitat Politécnica de Catalunya, Spain. He is currently a Full Professor with the University of Fortaleza (UNIFOR), Fortaleza, Brazil. His current research interests include machine learning, knowledge graphs, and complex networks.

Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) - ROR identifier: 00x0ma614